

[Open in Colab](#)

Prompt Engineering for RAG

In this notebook we show various prompt techniques you can try to customize your LlamaIndex RAG pipeline.

- Getting and setting prompts for query engines, etc.
- Defining template variable mappings (e.g. you have an existing QA prompt)
- Adding few-shot examples + performing query transformations/rewriting.

```
%pip install llama-index-llms-openai
%pip install llama-index-readers-file pymupdf
```

```
!pip install llama-index
```

```
import os
import openai
```

```
os.environ["OPENAI_API_KEY"] = "sk-..."
openai.api_key = os.environ["OPENAI_API_KEY"]
```

Setup

```
import logging
import sys

logging.basicConfig(stream=sys.stdout, level=logging.INFO)
logging.getLogger().addHandler(logging.StreamHandler(stream=sys.stdout))

from llama_index.core import VectorStoreIndex
from llama_index.core import PromptTemplate
from IPython.display import Markdown, display
```

```
INFO:numexpr.utils:Note: NumExpr detected 12 cores but "NUMEXPR_MAX_THREADS" not set, so enforcing
safe limit of 8.
Note: NumExpr detected 12 cores but "NUMEXPR_MAX_THREADS" not set, so enforcing safe limit of 8.
INFO:numexpr.utils:NumExpr defaulting to 8 threads.
NumExpr defaulting to 8 threads.
```

Load Data

```
!mkdir data
!wget --user-agent "Mozilla" "https://arxiv.org/pdf/2307.09288.pdf" -O "data/llama2.pdf"
```

```
mkdir: data: File exists
--2023-10-28 23:19:38-- https://arxiv.org/pdf/2307.09288.pdf
Resolving arxiv.org (arxiv.org)... 128.84.21.199
Connecting to arxiv.org (arxiv.org)|128.84.21.199|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13661300 (13M) [application/pdf]
```

Saving to: 'data/llama2.pdf'

data/llama2.pdf 100%[=====>] 13.03M 1.50MB/s in 10s

2023-10-28 23:19:49 (1.31 MB/s) - 'data/llama2.pdf' saved [13661300/13661300]

```
from pathlib import Path
from llama_index.readers.file import PyMuPDFReader
```

```
loader = PyMuPDFReader()
documents = loader.load(file_path="./data/llama2.pdf")
```

Load into Vector Store

```
from llama_index.core import VectorStoreIndex
from llama_index.llms.openai import OpenAI

gpt35_llm = OpenAI(model="gpt-3.5-turbo")
gpt4_llm = OpenAI(model="gpt-4")

index = VectorStoreIndex.from_documents(documents)
```

Setup Query Engine / Retriever

```
query_str = "What are the potential risks associated with the use of Llama 2 as mentioned in the context?"
```

```
query_engine = index.as_query_engine(similarity_top_k=2, llm=gpt35_llm)
# use this for testing
vector_retriever = index.as_retriever(similarity_top_k=2)
```

```
response = query_engine.query(query_str)
print(str(response))
```

The potential risks associated with the use of Llama 2, as mentioned in the context, include the generation of misinformation and the retrieval of information about topics such as bioterrorism or cybercrime. The models have been tuned to avoid these topics and diminish any capabilities they might have offered for those use cases. However, there is a possibility that the safety tuning of the models may go too far, resulting in an overly cautious approach where the model declines certain requests or responds with too many safety details. Users of Llama 2 and Llama 2-Chat need to be cautious and take extra steps in tuning and deployment to ensure responsible use.

Viewing/Customizing Prompts

First, let's take a look at the query engine prompts, and see how we can customize it.

View Prompts

```
# define prompt viewing function
def display_prompt_dict(prompts_dict):
    for k, p in prompts_dict.items():
        text_md = f"Prompt Key: {k}<br>" f"Text: {p}<br>"
        display(Markdown(text_md))
```

```
print(p.get_template())
display(Markdown("<br><br>"))
```

```
prompts_dict = query_engine.get_prompts()
```

```
display_prompt_dict(prompts_dict)
```

Prompt Key: response_synthesizer:text_qa_template

Text:

Context information is below.

{context_str}

Given the context information and not prior knowledge, answer the query.

Query: {query_str}

Answer:

Prompt Key: response_synthesizer:refine_template

Text:

The original query is as follows: {query_str}

We have provided an existing answer: {existing_answer}

We have the opportunity to refine the existing answer (only if needed) with some more context below.

{context_msg}

Given the new context, refine the original answer to better answer the query. If the context isn't useful, return the original answer.

Refined Answer:

Customize Prompts

What if we want to do something different than our standard question-answering prompts?

Let's try out the RAG prompt from [LangchainHub](#)

```
# to do this, you need to use the langchain object

from langchain import hub

langchain_prompt = hub.pull("rlm/rag-prompt")
```

One catch is that the template variables in the prompt are different than what's expected by our synthesizer in the query engine:

- the prompt uses `context` and `question`,
- we expect `context_str` and `query_str`

This is not a problem! Let's add our template variable mappings to map variables. We use our `LangchainPromptTemplate` to map to LangChain prompts.

```
from llama_index.core.prompts import LangchainPromptTemplate

lc_prompt_tmpl = LangchainPromptTemplate(
```

```

    template=langchain_prompt,
    template_var_mappings={"query_str": "question", "context_str": "context"},
)

query_engine.update_prompts(
    {"response_synthesizer:text_qa_template": lc_prompt_tmpl}
)

```

```

prompts_dict = query_engine.get_prompts()
display_prompt_dict(prompts_dict)

```



Prompt Key: response_synthesizer:text_qa_template

Text:

```

input_variables=['question', 'context'] messages=[HumanMessagePromptTemplate(prompt=PromptTemplate(
    input_variables=['question', 'context'], template="You are an assistant for question-answering tasks. Use the following pieces of retrieved context to answer the question. If you don't know the answer, just say that you don't know. Use three sentences maximum and keep the answer concise.\nQuestion: {question} \nContext: {context} \nAnswer:"))]]

```

Prompt Key: response_synthesizer:refine_template

Text:

```

The original query is as follows: {query_str}
We have provided an existing answer: {existing_answer}
We have the opportunity to refine the existing answer (only if needed) with some more context below.
-----
{context_msg}
-----

Given the new context, refine the original answer to better answer the query. If the context isn't useful, return the original answer.
Refined Answer:

```

Try It Out

Let's re-run our query engine again.

```

response = query_engine.query(query_str)
print(str(response))

```



The potential risks associated with the use of Llama 2 mentioned in the context include the generation of misinformation, retrieval of information about topics like bioterrorism or cybercrime, an overly cautious approach by the model, and the need for users to be cautious and take extra steps in tuning and deployment. However, efforts have been made to tune the models to avoid these topics and diminish any capabilities they might have offered for those use cases.

Adding Few-Shot Examples

Let's try adding few-shot examples to the prompt, which can be dynamically loaded depending on the query!

We do this by setting the `function_mapping` variable in our prompt template - this allows us to compute functions (e.g. return few-shot examples) during prompt formatting time.

As an example use case, through this we can coerce the model to output results in a structured format, by showing examples of other structured outputs.

Let's parse a pre-generated question/answer file. For the sake of focus we'll skip how the file is generated (tl;dr we used a GPT-4 powered function calling RAG pipeline), but the qa pairs look like this:

```
{"query": "<query>", "response": "<output_json>"}
```

We embed/index these Q/A pairs, and retrieve the top-k.

```
from llama_index.core.schema import TextNode

few_shot_nodes = []
for line in open("../llama2_qa_citation_events.jsonl", "r"):
    few_shot_nodes.append(TextNode(text=line))

few_shot_index = VectorStoreIndex(few_shot_nodes)
few_shot_retriever = few_shot_index.as_retriever(similarity_top_k=2)
```

```
import json

def few_shot_examples_fn(**kwargs):
    query_str = kwargs["query_str"]
    retrieved_nodes = few_shot_retriever.retrieve(query_str)
    # go through each node, get json object

    result_strs = []
    for n in retrieved_nodes:
        raw_dict = json.loads(n.get_content())
        query = raw_dict["query"]
        response_dict = json.loads(raw_dict["response"])
        result_str = f"""\
Query: {query}
Response: {response_dict}"""
        result_strs.append(result_str)
    return "\n\n".join(result_strs)
```

```
# write prompt template with functions
qa_prompt_tmpl_str = """\
Context information is below.
-----
{context_str}
-----

Given the context information and not prior knowledge, \
answer the query asking about citations over different topics. \
Please provide your answer in the form of a structured JSON format containing \
a list of authors as the citations. Some examples are given below.

{few_shot_examples}

Query: {query_str}
Answer: \
"""

qa_prompt_tmpl = PromptTemplate(
    qa_prompt_tmpl_str,
    function_mappings={"few_shot_examples": few_shot_examples_fn},
)
```

```
citation_query_str = (
    "Which citations are mentioned in the section on Safety RLHF?"
)
```

Let's see what the formatted prompt looks like with the few-shot examples function. (we fill in test context for brevity)

```
print(
    qa_prompt_tmpl.format(
        query_str=citation_query_str, context_str="test_context"
    )
)
```

Context information is below.

test_context

Given the context information and not prior knowledge, answer the query asking about citations over different topics.

Please provide your answer in the form of a structured JSON format containing a list of authors as the citations. Some examples are given below.

Query: Which citation discusses the impact of safety RLHF measured by reward model score distributions?

Response: {'citations': [{'author': 'Llama 2: Open Foundation and Fine-Tuned Chat Models', 'year': 24, 'desc': 'Impact of safety RLHF measured by reward model score distributions. Left: safety reward model scores of generations on the Meta Safety test set. The clustering of samples in the top left corner suggests the improvements of model safety. Right: helpfulness reward model scores of generations on the Meta Helpfulness test set.'}]}

Query: Which citations are mentioned in the section on RLHF Results?

Response: {'citations': [{'author': 'Gilardi et al.', 'year': 2023, 'desc': ''}, {'author': 'Huang et al.', 'year': 2023, 'desc': ''}]}

Query: Which citations are mentioned in the section on Safety RLHF?

Answer:

```
query_engine.update_prompts(
    {"response_synthesizer:text_qa_template": qa_prompt_tmpl}
)
```

```
display_prompt_dict(query_engine.get_prompts())
```

Prompt Key: response_synthesizer:text_qa_template

Text:

Context information is below.

{context_str}

Given the context information and not prior knowledge, answer the query asking about citations over different topics.

Please provide your answer in the form of a structured JSON format containing a list of authors as the citations. Some examples are given below.

{few_shot_examples}

Query: {query_str}

Answer:

Prompt Key: response_synthesizer:refine_template

Text:

The original query is as follows: {query_str}

We have provided an existing answer: {existing_answer}

We have the opportunity to refine the existing answer (only if needed) with some more context below

w.

```
-----
{context_msg}
-----
```

Given the new context, refine the original answer to better answer the query. If the context isn't useful, return the original answer.

Refined Answer:

```
response = query_engine.query(citation_query_str)
print(str(response))
```

```
{'citations': [{'author': 'Llama 2: Open Foundation and Fine-Tuned Chat Models', 'year': 24, 'desc': 'Safety RLHF'}, {'author': 'Bai et al.', 'year': 2022a, 'desc': 'RLHF stage'}, {'author': 'Bai et al.', 'year': 2022a, 'desc': 'adversarial prompts'}, {'author': 'Bai et al.', 'year': 2022a, 'desc': 'safety reward model'}, {'author': 'Bai et al.', 'year': 2022a, 'desc': 'helpfulness reward model'}, {'author': 'Bai et al.', 'year': 2022a, 'desc': 'safety tuning with RLHF'}]}
```

```
print(response.source_nodes[1].get_content())
```

Context Transformations - PII Example

We can also dynamically add context transformations as functions in the prompt variable. In this example we show how we can process the `context_str` before feeding to the context window - specifically in masking out PII (a step towards alleviating concerns around data privacy/security).

NOTE: You can do these as steps before feeding into the prompt as well, but this gives you flexibility to perform all this on the fly for any QA prompt you define!

```
from llama_index.core.postprocessor import (
    NERPIINodePostprocessor,
    SentenceEmbeddingOptimizer,
)
from llama_index.core import QueryBundle
from llama_index.core.schema import NodeWithScore, TextNode
```

```
pii_processor = NERPIINodePostprocessor(llm=gpt4_llm)
```

```
def filter_pii_fn(**kwargs):
    # run optimizer
    query_bundle = QueryBundle(query_str=kwargs["query_str"])

    new_nodes = pii_processor.postprocess_nodes(
        [NodeWithScore(node=TextNode(text=kwargs["context_str"]))],
        query_bundle=query_bundle,
    )
    new_node = new_nodes[0]
    return new_node.get_content()
```

```
qa_prompt_tmpl_str = (
    "Context information is below.\n"
    "-----\n"
    "{context_str}\n"
    "-----\n"
    "Given the context information and not prior knowledge, "
    "answer the query.\n"
```

```
    "Query: {query_str}\n"  
    "Answer: "  
)  
qa_prompt_tmpl = PromptTemplate(  
    qa_prompt_tmpl_str, function_mappings={"context_str": filter_pii_fn}  
)
```

```
query_engine.update_prompts(  
    {"response_synthesizer:text_qa_template": qa_prompt_tmpl}  
)
```

```
# take a look at the prompt  
retrieved_nodes = vector_retriever.retrieve(query_str)  
context_str = "\n\n".join([n.get_content() for n in retrieved_nodes])
```

```
print(qa_prompt_tmpl.format(query_str=query_str, context_str=context_str))
```

```
response = query_engine.query(query_str)  
print(str(response))
```